Because good things come in threes, here's yet another way to invoke methods dynamically: The eval method (and its variations such as class_eval, module_eval, and instance_eval) will parse and execute an arbitrary string of legal Ruby source code:

```
trane = %q{"John Coltrane".length}
miles = %q{"Miles Davis".sub(/iles/, '.')}

eval(trane) # => 13
eval(miles) # => "M. Davis"
```

When using eval, it can be helpful to state explicitly the context in which the expression should be evaluated, rather than using the current context. You obtain a context using binding at the desired point:

```
def get_a_binding
  val = 123
  binding
end

val = "cat"

the_binding = get_a_binding
eval("val", the_binding) # => 123
eval("val")              # => "cat"
```

The first eval evaluates val in the context of the binding *as it was* when the method get_a_binding was executing. In this binding, the variable val had a value of 123. The second eval evaluates eval in the top-level binding, where it has the value "cat".

Performance Considerations

As we've seen in this section, Ruby gives us several ways to invoke an arbitrary method of some object: send, call, and the various flavors of eval.

You may prefer to use any one of these techniques depending on your needs, but be aware that, as the following benchmark shows, eval is significantly slower than the others (or, for optimistic readers, send and call are significantly faster than eval).

```
require 'benchmark'
include Benchmark

test = "Stormy Weather"
m = test.method(:length)
n = 100000

bm(12) do |x|
  x.report("call") { n.times { m.call } }
  x.report("send") { n.times { test.send(:length) } }
  x.report("eval") { n.times { eval "test.length" } }
end
```

produces:

```
                 user     system      total        real
call         0.004662   0.000001   0.004663 (  0.004663)
send         0.005049   0.000037   0.005086 (  0.005084)
eval         0.210142   0.002401   0.212543 (  0.212553)
```